Programación Concurrente y Paralela

Mensajes y Canales

Hasta ahora vimos

- Hasta ahora, vimos diferentes maneras de comunicar threads de un mismo proceso utilizando memoria compartida.
- Ciertamente, vimos cómo la ejecución en *interleavings* indeseables puede generar resultados erróneos.

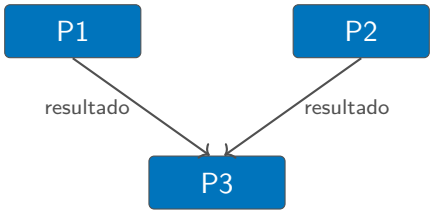
Alternativa

Eliminar la memoria compartida.

En principio esto no sólo elimina el problema, sino que también la posibilidad de comunicar distintos procesos ... ¿o no?

Ejemplo

Tres procesos donde el tercero necesita los resultados de los otros dos:



¿Cómo se comunican sin memoria compartida?

Mensajes

Mecanismo de sincronización que permite enviar y recibir datos a través de *canales* de comunicación.

Operaciones

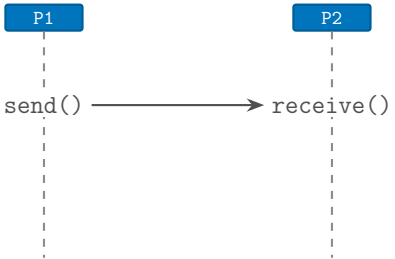
Sobre un canal `c`:

- `c.send(v)`: envía el valor `v` por el canal.
- `v = c.receive()`: recibe un valor del canal y lo asigna a la variable `v`.

¿Son operaciones bloqueantes?

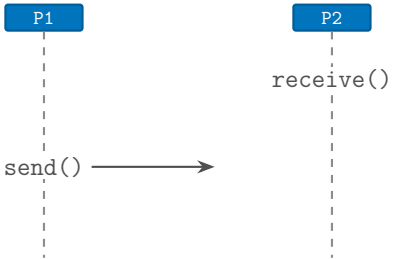
Depende del *modelo de comunicación*.

Pensemos en trazas I



Si el `send()` y el `receive()` suceden coordinadamente, el dato pasa directamente de un thread a otro, ninguno se bloquea

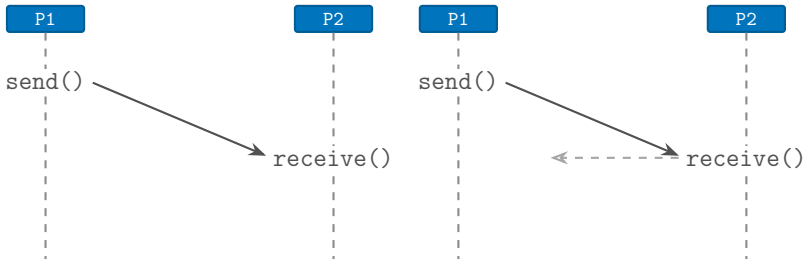
Pensemos en trazas II



¿Qué sucede si se ejecuta el `receive` antes? El proceso se bloquea, no tiene dato para seguir

Pensemos en trazas III

¿Qué sucede si se ejecuta el `send` antes?



Asincrónica

El mensaje se guarda en el canal y el proceso que hace `send` continúa su ejecución.

Sincrónica (*rendezvous*)

El `send` es bloqueante: el emisor espera al receptor.

Definición

Canales

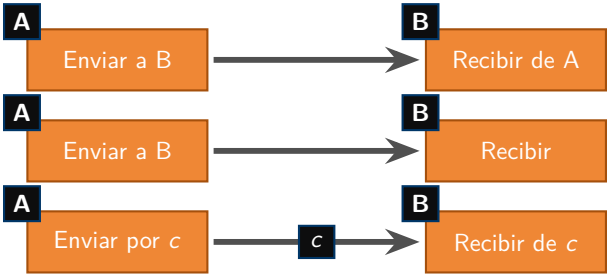
Un canal es un medio de comunicación entre procesos que permite hacer el intercambio de mensajes. Lo veremos como un tipo abstracto de datos ignorando los detalles de su implementación.

Los podemos ver como *buffers*:

- En teoría, canales asincrónicos pueden verse como buffers de capacidad no acotada.
- En la práctica se implementan con buffers de tamaño acotado.
- En la próxima sección los usaremos concretamente en Go, donde `make(chan T)` crea un canal sincrónico y `make(chan T, n)` crea uno asincrónico de capacidad `n`.

Identificación de procesos

¿Cómo se identifica el origen y el destino de un mensaje?



Identificación simétrica directa: A tiene un canal para enviar mensajes a B y uno para recibir mensajes de B (y viceversa).

Identificación asimétrica directa: A tiene un canal para enviar mensajes a B y uno para recibir mensajes (en general).

Identificación indirecta: los canales a usar no están preasignados, se declaran al principio o durante la ejecución.

En esta materia

- En nuestro pseudocódigo usamos **identificación indirecta**: los canales se declaran como variables y se los pasa a los procesos durante su creación.
- Los canales son objetos ("ciudadanos") de primera clase: se pueden pasar como parámetros, guardarse en estructuras... *y enviarse por otros canales*.
- En general trabajaremos con comunicación **asincrónica** (aunque Go ofrece ambas variantes).

Go

- Lenguaje desarrollado en Google (2009) por Robert Griesemer, Rob Pike y Ken Thompson.
- Compilado, con tipos estáticos, *garbage collection* y una sintaxis compacta.

- *Concurrencia nativa en el lenguaje*: goroutines y canales son primitivas del lenguaje (no bibliotecas).
- Inspirado en el formalismo CSP y en la experiencia de Hoare, Dijkstra y Pike con lenguajes como Newsqueak y Limbo.

Lema

"Don't communicate by sharing memory; share memory by communicating." — Rob Pike

Primer programa

```
package main

import "fmt"

func main() {
    var entero int = 5
    cadena := "una_cadena"
    fmt.Printf("El numero es %d y la cadena es %s\n", entero, cadena)
}
```

- `func` define funciones; `main` es el punto de entrada.
- `:=` declara e inicializa con inferencia de tipos: se puede escribir `x := 5` o también `var x int = 5`.
- `fmt.Println` imprime una cadena, `fmt.Printf` permite imprimir con formato.

Segundo programa

```
package main

import "fmt"

func main() {
    var entero int = 7
    for entero > 0 {
        fmt.Printf("Quedan %d\n", entero)
        entero--
    }
}
```

- El único bucle es `for` (no existe `while`)
- Las cláusulas de `for` e `if` no llevan paréntesis.
- `for {//codigo}` es el equivalente del `while(true)`

Goroutines

```
func main() {
    go decir("mundo") // corre de manera concurrente
    decir("hola")     // corre en main
}

func decir(s string) {
    for i := 0; i < 3; i++ {
        fmt.Println(s)
    }
}
```

- `go f()` lanza una *goroutine*: función que corre concurrentemente con el resto del programa.
- Muy livianas (pocos KB): el runtime de Go multiplexa las goroutines sobre pocos threads del SO (símil *project Loom* de Java).
- Cuando `main` termina, el programa termina: si no sincronizamos, las goroutines mueren sin correr.

Canales sincrónicos

```
func main() {
    c := make(chan int) // canal de enteros, sin buffer
    go func() {
        c <- 2 // send: bloquea hasta que alguien recibe
    }()
    x := <-c // receive: bloquea hasta que alguien envía
    fmt.Println(x)
}
```

- `ch <- v` envía `v` por el canal; `v := <-ch` recibe.
- Un canal sin buffer es *sincrónico* (*rendezvous*): send y receive se encuentran y se bloquean mutuamente hasta el encuentro.
- Un receive de `nil` (o send sobre `nil`) bloquea para siempre.

Canales bufferizados

```
c := make(chan int, 2) // buffer de capacidad 2

c <- 1 // no bloquea: hay lugar
c <- 2 // no bloquea: último lugar libre
c <- 3 // bloquea: buffer lleno
```

- Un canal con buffer de tamaño n es *asincrónico hasta agotar la capacidad*: el send no bloquea mientras haya lugar.
- Si un thread recibe un mensaje del buffer lleno, libera un lugar, permitiendo el envío bloqueado (se hace en orden FIFO)
- El receive de un canal vacío bloquea igual que antes.
- `make(chan T)` es equivalente a `make(chan T, 0)`

En términos de la teoría

Sincrónico $\approx$ *rendezvous*; bufferizado $\approx$ canal asincrónico con buffer acotado de capacidad n.

select

```
select {
case v := <-c1:          // espera a recibir de c1
    fmt.Println(v)
case c2 <- x:            // espera a que alguien reciba x de c2
    fmt.Println("enviado")
case <-time.After(3 * time.Second):
    fmt.Println("timeout")
}
```

- Espera sobre varias operaciones de canales a la vez; ejecuta exactamente un `case` (Si hay varios `case` listos, elige uno **uniformemente al azar**).
- Si ninguno está listo, bloquea (se puede poner una cláusula `default` que nunca bloquea).
- `time.After` crea un canal donde llegará un mensaje en el tiempo fijado.
- `select {}` bloquea para siempre.

Mensajes con estructura

Los canales son *objetos de primera clase*: se pasan como parámetros y se envían por otros canales.

```
type Req struct {
    X     int         // el dato del pedido
    Reply chan int    // canal por donde responder
}

func Srv(c chan Req) {
    req := <-c
    req.Reply <- f(req.X)
}

func Cliente(c chan Req) {
    req := Req{X: 5, Reply: make(chan int)}
    c <- req
    y := <-req.Reply
}
```

- El patrón *canal de respuesta incluido en el mensaje* es la forma estándar de comunicarse con un servicio en Go.

Retomando el ejemplo en Go

- Recordar: P3 computa en función de los resultados de P1 y P2.

```
func main() {
    c1 := make(chan int)
    c2 := make(chan int)
    go P1(c1) // computa en función de un X, envía el resultado
    go P2(c2) // computa en función de un Y, envía el resultado
    r1 := <-c1
    r2 := <-c2
    fmt.Println(r1 + r2)
}

func P1(c chan int) { c <- costoX() }
func P2(c chan int) { c <- costoY() }
```

- Estos canales son *sincrónicos*: el envío de cada resultado espera a que P3 esté recibiéndolo.

¿Y si P3 les pasa X e Y inicialmente?

```
func main() {
    c1 := make(chan int)
    c2 := make(chan int)
    go P1(c1) // recibe x, computa, envía r1 por c1
    go P2(c2) // recibe y, computa, envía r2 por c2
    c1 <- X
    c2 <- Y
    r1, r2 := <-c1, <-c2
}
func P1(c chan int) {
    x := <-c
    c <- costo(x)
}
```

¿Funciona siempre?

Con canales sincrónicos **sí:** cada `send` de P3 se tiene que corresponder con su `receive` y P3 no puede leer su propio mensaje.

¿Y si P3 les pasa X e Y inicialmente?

```
func main() {
    c1 := make(chan int, 1)
    c2 := make(chan int, 1)
    go P1(c1) // recibe x, computa, envía r1 por c1
    go P2(c2) // recibe y, computa, envía r2 por c2
    c1 <- X
    c2 <- Y
    r1, r2 := <-c1, <-c2
}
func P1(c chan int) {
    x := <-c
    c <- costo(x)
}
```

¿Funciona siempre?

Con canales asincrónicos **no:** si P1 y P2 no llegan a hacer sus `receive`, P3 lee en `r1` y `r2` los valores `X` e `Y` que acaba de enviar!

Solución: canales separados por dirección

```
c1, c2, c3, c4 := make(chan int,1), make(chan int,1), make(chan int,1),
    make(chan int,1)

func P1(pedidos, respuestas chan int) {
    x := <-pedidos
    respuestas <- costo(x)
}

func P3() {
    c1 <- X
    c2 <- Y
    r1 := <-c3
    r2 := <-c4
}
```

- Una forma sencilla de evitar problemas es pensar cada canal como una *dirección*: pedidos hacia el servidor, respuestas hacia el cliente.

Pregunta

¿Qué diferencia hay entre estas dos formas de escribir P3?

```
c1 <- X
c2 <- Y
r1 := <-c3
r2 := <-c4
```

```
c1 <- X
r1 := <-c3
c2 <- Y
r2 := <-c4
```

Respuesta

La segunda no aprovecha la concurrencia: espera el resultado de P1 antes de iniciar el cómputo de P2.

Ejercicios

Ejercicio I

Escribir dos procesos A y B. El proceso A genera un número aleatorio y se lo envía al proceso B, que al recibirlo lo muestra por pantalla.

```
func A(c chan int) { c <- rand.Intn(100) } //genera número entre 0 y 99

func main() { //rol de proceso B
    c := make(chan int)
    go A(c)
    fmt.Println(<-c)
}
```

Ejercicios

Ejercicio II

Extender: B lee un número n del usuario y se lo envía a A, quien genera la cantidad de números pedida. B muestra todos los números por pantalla.

```
func A(ped, resp chan int) {
    n := <-ped
    for i := 0; i < n; i++ {
        resp <- rand.Intn(100)
    }
}
```

```
func main() {  //rol de proceso B
    p, r := make(chan int), make(
    chan int)
    go A(p, r)
    n := leerEntero()
    p <- n
    for i := 0; i < n; i++ {
        fmt.Println(<-r)
    }
}
```

Pregunta

¿Cómo extendemos para atender pedidos (de múltiples instancias de B) de manera concurrente?

- Necesitamos un proceso que reciba el pedido y *delegue en otro* la generación de los números.

```
func A(pedidos, respuestas chan int) {
    for {
        n := <-pedidos
        go func(n int, r chan int) { // una goroutine por pedido
            for i := 0; i < n; i++ {
                r <- rand.Intn(100)
            }
        }(n, respuestas)
    }
}
```

¿Está bien esta solución?

¿Qué pasa si hay varios clientes B enviando pedidos?

- Por lo pronto, no nos cambia: Las respuestas se pueden mezclar pero es indistinto
- Esto cambia si, por ejemplo, cada B pone un valor tope para los números aleatorios que quiere. Ahora...

Problema

Todos comparten `respuestas`: las respuestas de un cliente pueden llegarle a otro.

- Necesitamos un *canal de respuesta para cada cliente*.
- Dependiendo del protocolo, el canal puede proponerlo el cliente o elegirlo el servidor.

Solución: el canal viaja en el mensaje

```
type Request struct {
    N, M  int  //N numeros con tope M
    Reply chan int
}
func A(c chan Request) {
    for {
        req := <- c
        go func(req Request) {
            for i := 0; i < req.N; i++ {
                req.Reply <- rand.Intn(req.M)
            }
        }(req)
    }
}
func B(c chan Request) {
    req := Request{N: leerEntero(), M: leerEntero(),
                   Reply: make(chan int)}
    c <- req
    for i := 0; i < req.N; i++ {
        fmt.Println(<-req.Reply)
    }
}
```

Semáforo con canales

¿Cómo podemos implementar un semáforo usando canales?

- Manera directa: El semáforo *es un canal*. Invariante: La cantidad de mensajes en el canal es la cantidad de permisos del semáforo.
- Si queremos una estructura con una interfaz (y no modelar cada permiso como un objeto), podemos tener una goroutine que es el semáforo, con canales para hacerle `acquire` y `release`.
- Dentro de la goroutine, solo tomo los mensajes en el canal para `acquire` cuando hay suficientes permisos en el semáforo.

Semáforo con canales

```
func NewSem(n int) (acq, rel chan struct{}) { //struct{} : tipo unitario
    a, r := make(chan struct{}), make(chan struct{})
    go func(p int) {
        for {
            if p == 0 {
                <-r // sólo puede llegar un release
                p = 1
            } else {
                select {
                case <-a : p-- // un acquire se satisface
                case <-r : p++ // un release se satisface
                }
            }
        }
    }(n)
    return a, r
}
```

Semáforo con canales: uso

```
func main() {
    acq, rel := NewSem(1)

    for i := 0; i < 3; i++ {
        go func(i int, acqch chan struct{}, relch chan struct{}) {
            acqch <- struct{}{}
            fmt.Println("pase", i) // sección crítica
            relch <- struct{}{}
        }(i,acq,rel)
    }
}
```

- Observar: `acquire` y `release` no son send/receive "opuestos", ambos son send.
- Con `select`, la goroutine de estado ofrece a la vez `acquire` y `release`: *elección no determinista*.

Servidor con estado: cuenta bancaria

Así como el semáforo, podemos crear una goroutine que sea un objeto compartido con todo tipo de estado y operaciones.

```
type Op interface{} //Op puede ser cualquier tipo
type Put struct{ F float64 }
type Get struct{ F float64; Reply chan float64 }
func Cuenta(in chan Op) {
    bal := 0.0
    for {
        switch op := (<-in).(type) { //Me fijo cual tipo es
        case Put:
            bal += op.F // poner no da respuesta
        case Get:
            bal -= op.F // permite pasarse
            op.Reply <- bal
        }
    }
}
```

- `Put` no contesta (*fire and forget*); `Get` responde por el canal de respuesta indicado.
- La cuenta permite `Get` con saldo negativo.

Filósofos comensales

La solución (*naive*) para los filósofos comensales con mensajes propuesta por Ben-Ari usa threads para los tenedores.

```
func Tenedor(forks chan struct{}) {
    for {
        forks <- struct{}{}
        <-forks
    }
}

func Filosofo(i int, forks []chan struct{}) {
    izq, der := forks[i], forks[(i+1)%len(forks)]
    for {
        // pensar
        <-izq
        <-der
        fmt.Println("filósofo", i, "come")
        izq <- struct{}{}
        der <- struct{}{}
    }
}
```

Filósofos comensales

```
func main() {
    forks := make([]chan struct{}, 5)
    for i := range forks {
        forks[i] = make(chan struct{})
        go Tenedor(forks[i])
    }
    for i := 0; i < 5; i++ {
        go Filosofo(i, forks)
    }
    select {}
}
```

¿Cómo logramos resolver el deadlock de la solución *naive*?

- Rompiendo la simetría (haciendo que un filósofo invierta el orden en tomar los tenedores)
- Agregando un semáforo implementado por mensajes (como vimos)
- Agregando un thread que gestione la toma de los tenedores (y se fije cuáles hay, recordando cuáles entregó y cuáles ya se devolvieron)

Algunas notas

- Este es sólo un pantallazo general de concurrencia por medio de mensajes usando Go, el lenguaje ofrece muchas más posibilidades
- Go no es un lenguaje de comunicación por pasajes de mensajes *puro*, se puede compartir memoria!
- En la práctica muchas soluciones concurrentes combinan ambos mecanismos de comunicación (incorporando *Locks* o *Mutexes* por ejemplo)

Álgebras de procesos

- En la materia venimos construyendo herramientas prácticas: semáforos, monitores, STM, canales.
- Vimos herramientas como los diagramas de transición, pero principalmente razonamos de manera informal sobre *trazas* e *interleavings*.

- *Las álgebras de procesos* son modelos matemáticos donde:
  - un sistema concurrente es un *término* de un álgebra;
  - el comportamiento se describe por las *interacciones* (eventos) que el sistema puede tener con su entorno.
- Permiten especificar, refinar y razonar formalmente.

- Típicamente, modelan la comunicación entre threads por medio de mensajes, ya que finalmente las variables compartidas se pueden pensar como un thread que atiende pedidos.

Algunas álgebras de procesos

- Podemos mencionar CSP (*Communicating Sequential Processes*), desarrollado por Tony Hoare, y de gran influencia para múltiples lenguajes (incluído Go). En CSP existen etiquetas a las que un conjunto de (dos o más) threads deben llegar para sincronizarse.
- En desarrollo paralelo con CSP, Robin Milner desarrollaba CCS (*Calculus of Communicating Systems*), donde las etiquetas de sincronización son estrictamente de a pares de procesos, el que envía y el que recibe.
- Finalmente, tenemos el π-Cálculo, también desarrollado por Milner, Parrow y Walker, evolucionando sobre CCS. El elemento principal que se agrega es el hecho de que estas etiquetas (nombres) *son objetos que pueden ser enviados* en otros nombres, llegando a la topología dinámica que vemos en Go.

π-cálculo: nombres y movilidad

Sintaxis básica

$$P ::= \overline{x}\langle y\rangle.P \mid x(y).P \mid P \mid Q \mid (\nu x)\, P \mid !P \mid \mathbf{0}$$

- Los *nombres* (x, y) son los canales: la única primitiva.
- $\overline{x}\langle y\rangle.P$: envía y por x y continúa como P (salida).
- $x(y).P$: recibe y por x (input; y queda *bindeado*).
- (νx): declara un nombre nuevo (privado).
- $!P$: replicación (un servidor eterno).

π-cálculo: el pedido con canal de respuesta

El cliente manda su pedido junto con un canal de respuesta:

$$CLIENTE = (\nu a)\,\overline{x}\langle a\rangle.a(z).\,\overline{p}\langle z\rangle.\mathbf{0}$$
$$SERVIDOR = !x(a).\overline{a}\langle v\rangle.\mathbf{0}$$

```
type Req struct {
    X      int
    Reply chan int // a: el canal que viaja en el mensaje
}
```

- *Movilidad*: los nombres viajan en los mensajes; la topología de comunicación cambia en tiempo de ejecución.
- A diferencia de CSP, la estructura de comunicación es dinámica — así lo modelamos cuando delegamos con go.

Referencias

- C. A. R. Hoare. *Communicating Sequential Processes*. Prentice Hall, 1985. (Disponible gratis en `usingcsp.com`.)
- R. Milner. *Communicating and Mobile Systems: the π-calculus*. Cambridge University Press, 1999.
- R. Pike. *Concurrency is not Parallelism*. Charla, 2012.
- *The Go Memory Model* y *The Go Programming Language Specification* (go.dev).